Module 1 Assignment: Problem Statement

Total Marks: 100

This assignment introduces and reinforces foundational supervised and unsupervised machine learning techniques: Linear Regression, Logistic Regression, Support Vector Classification, Bagging (Bootstrap Aggregating), and K-Means Clustering, through hands-on implementation using both library functions (scikit-learn) and, in one section, an algorithm built entirely from scratch. Two well-known benchmark datasets are used throughout: the California Housing dataset (regression) and the Iris dataset (classification and clustering).

Unless a question explicitly states otherwise, learners are permitted to use standard library functions (NumPy, pandas, scikit-learn, matplotlib, seaborn) to complete each task.

Structure of the Assignment

Part 1: Linear Regression [10 marks]

Dataset: California Housing dataset (sklearn.datasets.fetch_california_housing).

Objective: Build intuition for simple linear regression by modeling the relationship between a single predictor and a continuous target.

Tasks:

1. **Q1 (2 marks):** Print the feature names of the dataframe.
2. **Q2 (2 marks):** Convert MedHouseVal feature to input, and MedInc feature to target, and then print the shapes of each (no need to have validation split or test split for this question).
3. **Q3 (2 marks):** Visualize the samples as a 2D scatter plot.
4. **Q4 (2 marks):** Fit a linear regressor on the data. Print the resultant slope and intercept.
5. **Q5 (2 marks):** Visualize the linear regression result by plotting the regression line on top of the 2D scatter plot of MedInc and MedHouseVal.

Part 2: Logistic Regression [10 marks]

Dataset: Iris dataset (sklearn.datasets.load_iris), reduced to two classes.

Objective: Apply logistic regression to a binary classification problem and evaluate it on a held-out test set.

Tasks:

1. **Q6 (2 marks):** Print the input and output features of 5 different samples from the data.
2. **Q7 (2 marks):** Visualize the features using a pairplot.
3. **Q8 (2 marks):** Convert the labels into integer class values (setosa = 0, versicolor = 1, virginica = 2) and then remove the middle class (versicolor). Change the label for virginica from 2 to 1.
4. **Q9 (2 marks):** Split the data into x (input) and y (target). Further split x and y into training and testing subsets, with 80% samples for training and 20% samples for testing.
5. **Q10 (2 marks):** Train a logistic regression model on the training subset. Print the weights and the bias. Predict the outputs using testing subset and print the resultant accuracy.

Part 3: Support Vector Classifier [20 marks]

Dataset: Iris dataset, restricted to two features (petal length, petal width).

Objective: Understand how the regularization parameter C in a linear SVM affects the decision boundary, margin width, and generalization.

Tasks:

1. **Q11 (2 marks):** Take only two classes as targets, by removing all the samples belonging to the class "Verginica". Print any 5 samples from the data.
2. **Q12 (2 marks):** Plot the input features as a 2D scatter plot. The two classes should be shown with different colors.
3. **Q13 (2 marks):** Split the data into x (input) and y (target). Further split x and y into training and testing subsets, with 80% samples for training and 20% samples for testing.
4. **Q14 (7 marks):** Train a linear support vector classifier (with C=1 and hinge loss) on the training data. Plot the samples from the whole dataset, with decision boundary and margins (the two classes should have different colors). Predict on test set and print the resultant accuracy.
5. **Q15 (7 marks):** Train a linear support vector classifier (with C=1000 and hinge loss) on the training data. Plot the samples from the whole dataset, with decision boundary and margins (the two classes should have different colors). Predict on test set and print the resultant accuracy.

Part 4: Bagging [30 marks]

Dataset: California Housing dataset, reformatted into a labeled feature/target DataFrame.

Objective: Implement the core idea behind Bootstrap Aggregating (Bagging): training multiple base learners on bootstrap-resampled subsets of the training data and averaging their predictions and empirically study how ensemble size affects error.

Tasks:

1. **Q16 (5 marks):** Write a method to get the random indices from the train set and then use the random indices to create a variation of the training set, where some samples are randomly repeated while others are not included. Split the data into x (input) and y (target), each split into training and testing subsets, with 80% samples for training and 20% samples for testing.
2. **Q17 (3 marks):** Train a linear regression model on the training set with random indices and print the mean squared error of the predictions on the test set.
3. **Q18 (4 marks):** Fit a second linear regression model on a new variation of the training dataset with a new set of random indices and print the mean squared error of the average of the predictions of the two linear regression models (this model and the model trained in the previous question).
4. **Q19 (4 marks):** Fit a third linear regression model on a new variation of the training dataset with a new set of random indices and print the mean squared error of the average of the predictions of the three linear regression models (this model and the model trained in the previous two questions).
5. **Q20 (4 marks):** Fit a fourth linear regression model on a new variation of the training dataset with a new set of random indices and print the mean squared error of the average of the predictions of the four linear regression models (this model and the model trained in the previous three questions).
6. **Q21 (10 marks):** Loop through using 2, 4, 6 ... 30 models and train the models with variations of the training dataset. Print the test mean squared error (take the average mean squared error of the corresponding 2, 4, 6 ... 30 models). Finally, plot the training loss and testing loss for all the cases (2, 4, 6 ... 30 models).

Part 5: K-Means Clustering (From Scratch) [30 marks]

Dataset: Iris dataset, restricted to two features (petal length, petal width).

Objective: Implement the K-Means clustering algorithm from first principles (without sklearn.cluster.KMeans), including centroid assignment, centroid updates, convergence, the elbow method for choosing k, and final cluster visualization.

Tasks:

1. **Q22 (3 marks):** Write a method that takes a data point and a list of centroids, calculates the Euclidean distance between the point and each centroid, and returns the index of the closest centroid.
2. **Q23 (3 marks):** Write a method that takes a dictionary mapping cluster IDs to lists of 2D data points, calculates the new centroid for each cluster by averaging its points, and returns the updated centroids as a NumPy array.
3. **Q24 (3 marks):** Write a method that takes a random seed and an integer "p", sets the seed for reproducibility, and randomly selects "p" initial centroids from a globally defined points dataset, returning them as a NumPy array.
4. **Q25 (3 marks):** Write a method that calculates the Sum of Squared Errors (SSE) for a clustering model. The function should take an array of centroids, a dictionary mapping cluster indices to lists of data points, and the total number of clusters "k", and return the sum of the squared Euclidean distances between every point and its assigned centroid.
5. **Q26 (6 marks):** Write a method that implements the main K-means clustering loop. The function should take the number of clusters "p", a list of data points, and an array of initial centroids seed. It must iteratively assign each point to its nearest centroid, compute the new centroids, and repeat this process until convergence (when the centroids no longer change), ultimately returning the final centroids and a dictionary of the cluster assignments. Whenever K-means needs to be implemented for any of the following questions, this K-means implementation from scratch should be used.
6. **Q27 (6 marks):** Write a Python script that iterates over cluster counts from 1 to 4, executes the K-means algorithm using a fixed random seed of 42 for initialization, computes the Sum of Squared Errors (SSE) for each configuration, and returns the list of SSE values. Then plot the sum of squared distance in the y-axis and number of clusters in the x-axis.
7. **Q28 (6 marks):** Use the best number of clusters from the elbow method and visualize the data after applying K-means on the data using a 2D scatter plot. The two classes should be differently colored, and the centroids should also be shown.

Summary of Deliverables

Part	Topic	Marks	Key Skills Assessed
1	Linear Regression	10	Data loading, single-feature regression, visualization
2	Logistic Regression	10	Binary classification, train/test split, model evaluation
3	Support Vector Classifier	20	Margin-based classification, effect of regularization (C), decision boundary visualization
4	Bagging	30	Bootstrap resampling, ensemble averaging, bias-variance tradeoff analysis
5	K-Means Clustering	30	Unsupervised learning implemented from scratch, elbow method, convergence, cluster visualization
Total		**100**	

Learning Outcomes

By the end of this assignment, learners will be able to independently implement and apply the core supervised and unsupervised learning techniques covered in Module 1: linear regression, logistic regression, support vector classification, bagging, and K-means clustering, using both library implementations and, for K-means, an implementation built entirely from scratch, on standard benchmark datasets.

Submission Guidelines

1. **Save a Copy:** Open the provided notebook and save a copy to your own Google Drive before starting.
2. **Fill the Marked Cells:** Complete every cell containing ### YOUR CODE STARTS HERE ### ... ### YOUR CODE ENDS HERE ### with your own code, thereby solving every question that carries a mark value.
3. **Provided Cells:** There are some cells that should be kept unaltered. Only write code between "### YOUR CODE STARTS HERE ###" and "### YOUR CODE ENDS HERE ###" comments.
4. **Execute All Cells:** Before submitting, choose "Runtime", then "Restart and run all", so the notebook runs from top to bottom without errors.

5. **Download and Submit:** Download the completed notebook as an .ipynb file and submit it on the designated platform.
6. **Academic Integrity:** All code and written answers must be your own work.